# SKILL– 3

# Working with HQL - Sorting, Pagination & Aggregates

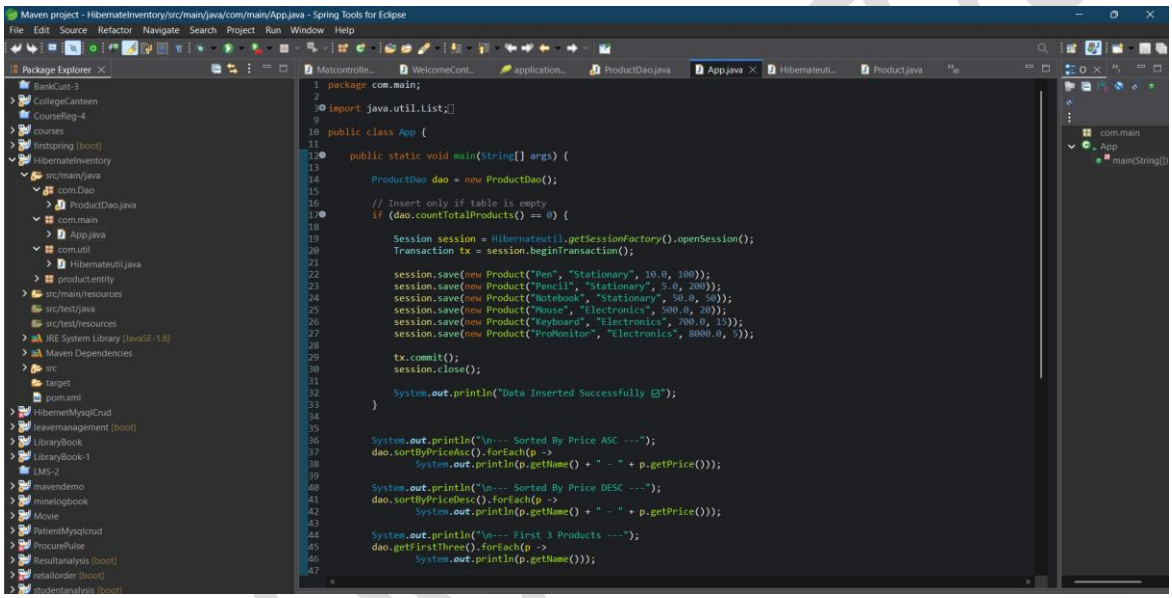
T. Amrutha Hasini

2400030474

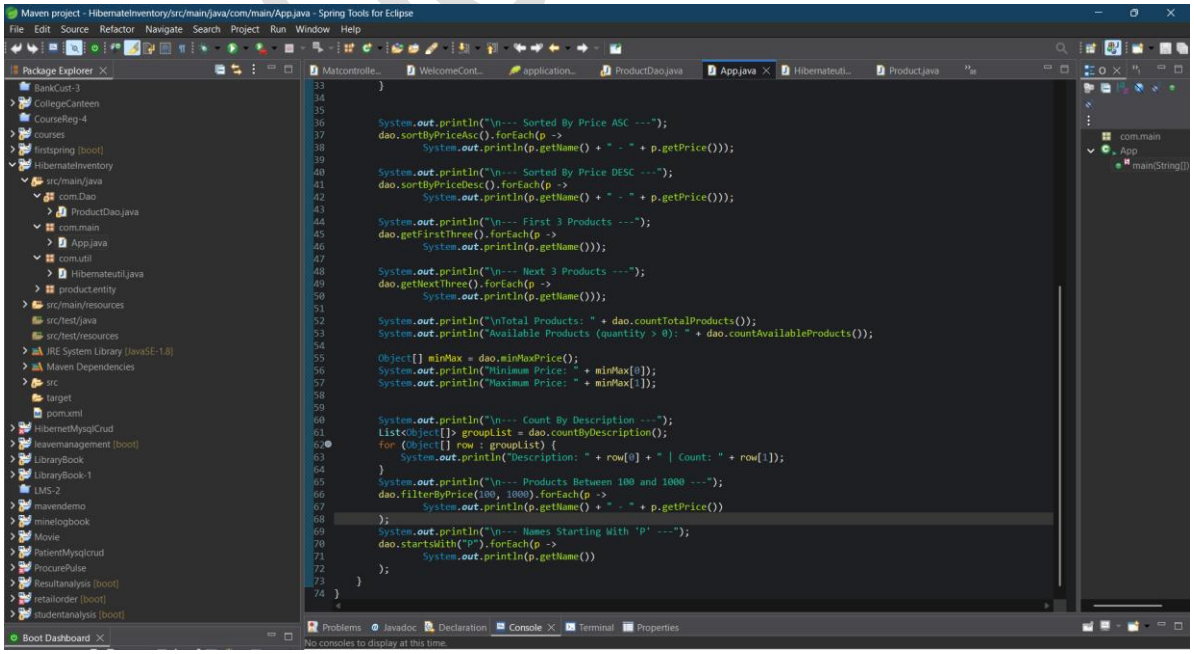
FSAD, Sec 4

## CODE:

App.Java:

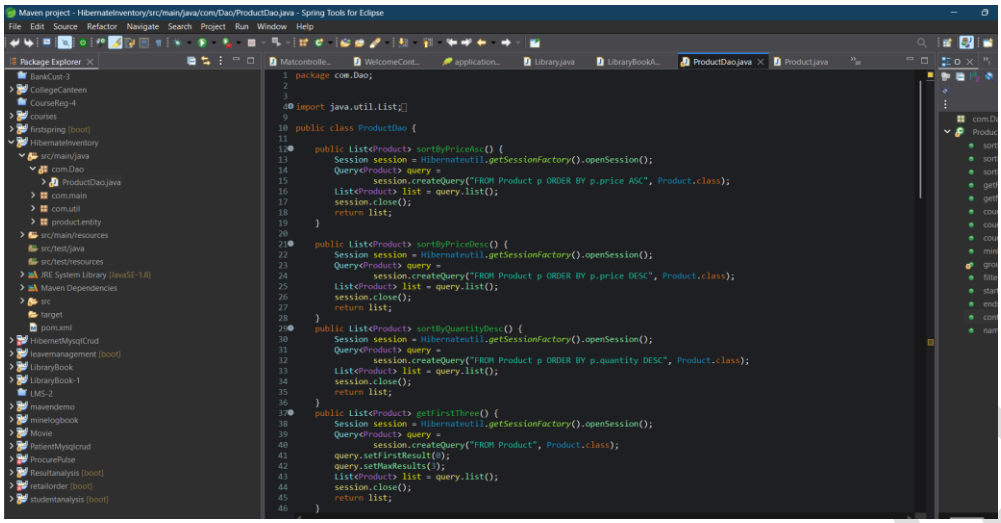


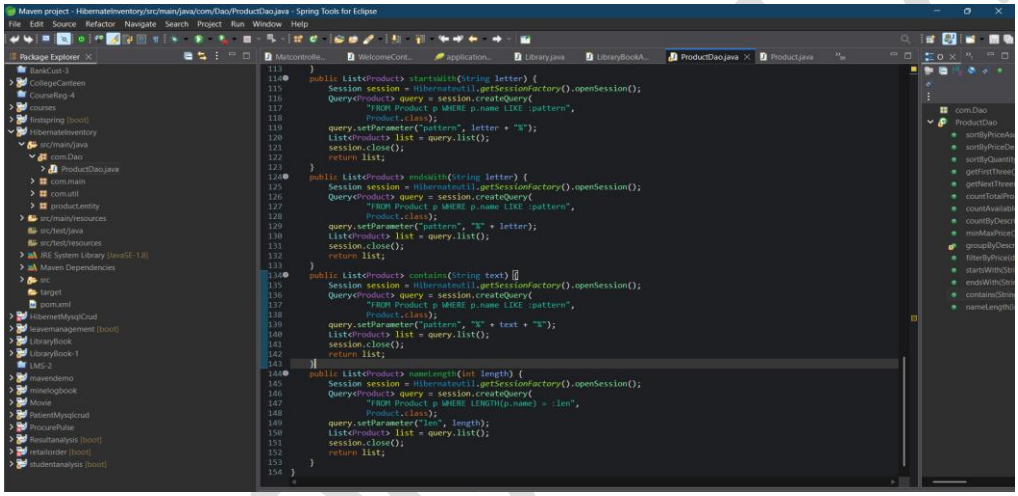
```java
package com.main;

import java.util.List;

public class App {

    public static void main(String[] args) {

        ProductDao dao = new ProductDao();

        // Insert only if table is empty
        if (dao.countTotalProducts() == 0) {

            Session session = Hibernateutil.getSessionFactory().openSession();
            Transaction tx = session.beginTransaction();

            session.save(new Product("Pen", "Stationary", 10.0, 100));
            session.save(new Product("Pencil", "Stationary", 5.0, 200));
            session.save(new Product("Notebook", "Stationary", 50.0, 50));
            session.save(new Product("Mouse", "Electronics", 500.0, 20));
            session.save(new Product("Keyboard", "Electronics", 700.0, 15));
            session.save(new Product("ProMonitor", "Electronics", 8000.0, 5));

            tx.commit();
            session.close();

            System.out.println("Data Inserted Successfully ");
        }


        System.out.println("\n--- Sorted By Price ASC ---");
        dao.sortByPriceAsc().forEach(p ->
                System.out.println(p.getName() + " - " + p.getPrice()));

        System.out.println("\n--- Sorted By Price DESC ---");
        dao.sortByPriceDesc().forEach(p ->
                System.out.println(p.getName() + " - " + p.getPrice()));

        System.out.println("\n--- First 3 Products ---");
        dao.getFirstThree().forEach(p ->
                System.out.println(p.getName()));
```



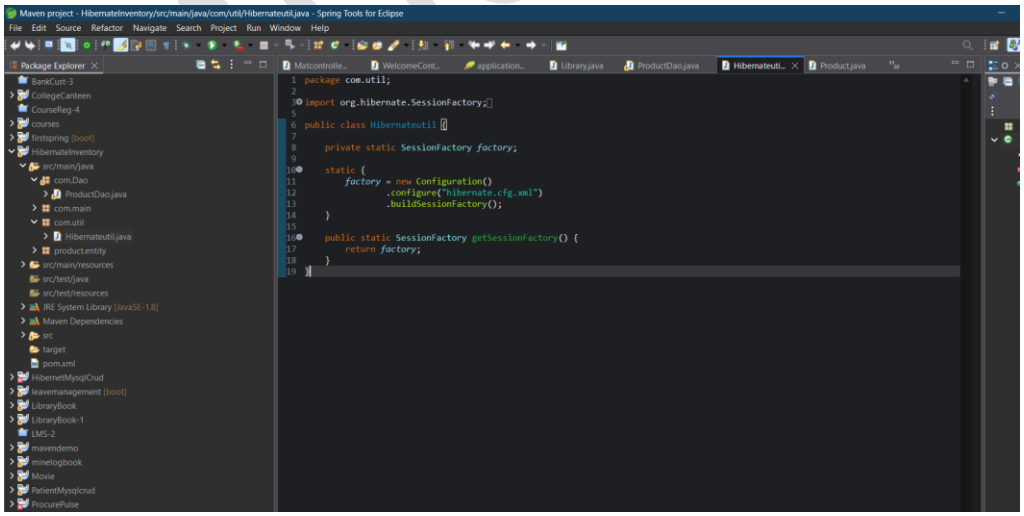
```java
        }


        System.out.println("\n--- Sorted By Price ASC ---");
        dao.sortByPriceAsc().forEach(p ->
                System.out.println(p.getName() + " - " + p.getPrice()));

        System.out.println("\n--- Sorted By Price DESC ---");
        dao.sortByPriceDesc().forEach(p ->
                System.out.println(p.getName() + " - " + p.getPrice()));

        System.out.println("\n--- First 3 Products ---");
        dao.getFirstThree().forEach(p ->
                System.out.println(p.getName()));

        System.out.println("\n--- Next 3 Products ---");
        dao.getNextThree().forEach(p ->
                System.out.println(p.getName()));

        System.out.println("\nTotal Products: " + dao.countTotalProducts());
        System.out.println("Available Products (quantity > 0): " + dao.countAvailableProducts());

        Object[] minMax = dao.minMaxPrice();
        System.out.println("Minimum Price: " + minMax[0]);
        System.out.println("Maximum Price: " + minMax[1]);


        System.out.println("\n--- Count By Description ---");
        List<Object[]> groupList = dao.countByDescription();
        for (Object[] row : groupList) {
            System.out.println("Description: " + row[0] + " | Count: " + row[1]);
        }
        System.out.println("\n--- Products Between 100 and 1000 ---");
        dao.filterByPrice(100, 1000).forEach(p ->
                System.out.println(p.getName() + " - " + p.getPrice())
        );
        System.out.println("\n--- Names Starting With 'P' ---");
        dao.startsWith("P").forEach(p ->
                System.out.println(p.getName())
        );
    }
}
```

PRODUCT DAO:

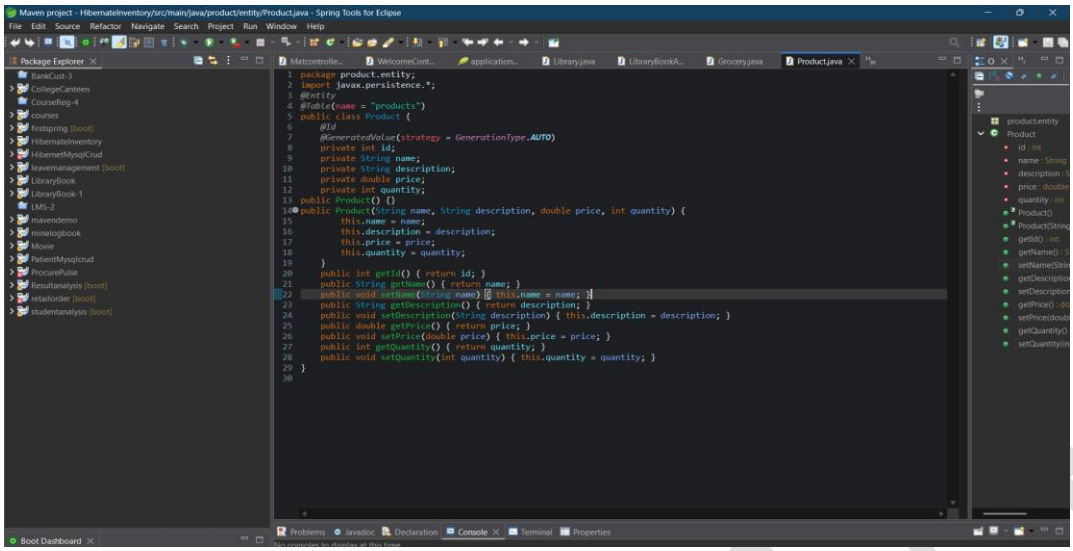



Hiberrnate Util:



PRODUCT ENTITY:



```java
package product.entity;
import javax.persistence.*;
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;
    private String name;
    private String description;
    private double price;
    private int quantity;
public Product() {}
public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

# OUTPUTS:

```
--- Sorted By Price ASC ---
Pencil - 5.0
Pen - 10.0
Notebook - 50.0
Mouse - 500.0
Keyboard - 700.0
ProMonitor - 8000.0

--- Sorted By Price DESC ---
ProMonitor - 8000.0
Keyboard - 700.0
Mouse - 500.0
Notebook - 50.0
Pen - 10.0
Pencil - 5.0

--- First 3 Products ---
Pen
Pencil
Notebook

--- Next 3 Products ---
Mouse
Keyboard
ProMonitor
```

```
Pen
Pencil
Notebook

--- Next 3 Products ---
Mouse
Keyboard
ProMonitor

Total Products: 6
Available Products (quantity > 0): 6
Minimum Price: 5.0
Maximum Price: 8000.0

--- Count By Description ---
Description: Stationary | Count: 3
Description: Electronics | Count: 3

--- Products Between 100 and 1000 ---
Mouse - 500.0
Keyboard - 700.0

--- Names Starting With 'P' ---
Pen
Pencil
ProMonitor
```

Git repo: - [https://github.com/AmruthaHasini-KLU/skill-3.git](https://github.com/AmruthaHasini-KLU/skill-3.git)